

CIS520 Project 4: One Program, Three Ways

Group 04: Jackson Seim, Tannyr Singleterry, Jacob Wuthnow

1) Experimental Configuration

All experiments were done on the Kansas State University's Beocat super cluster. We used the slurm batch scheduler. Jobs (at least this first part) were constrained to mole class nodes to keep hardware consistent.

2) What We Are Measuring

We aim to show the following performance analysis guidelines:

- Scalability: How does wall time improve as we add threads from 1 to 20?
- Parallel efficiency: Are threads being used effectively, or are they mostly idle?
- RAM usage: Does memory consumptions change with thread count or memory budget?
- I/O vs. computer balance: Is the bottleneck in the file reading or character scanning?

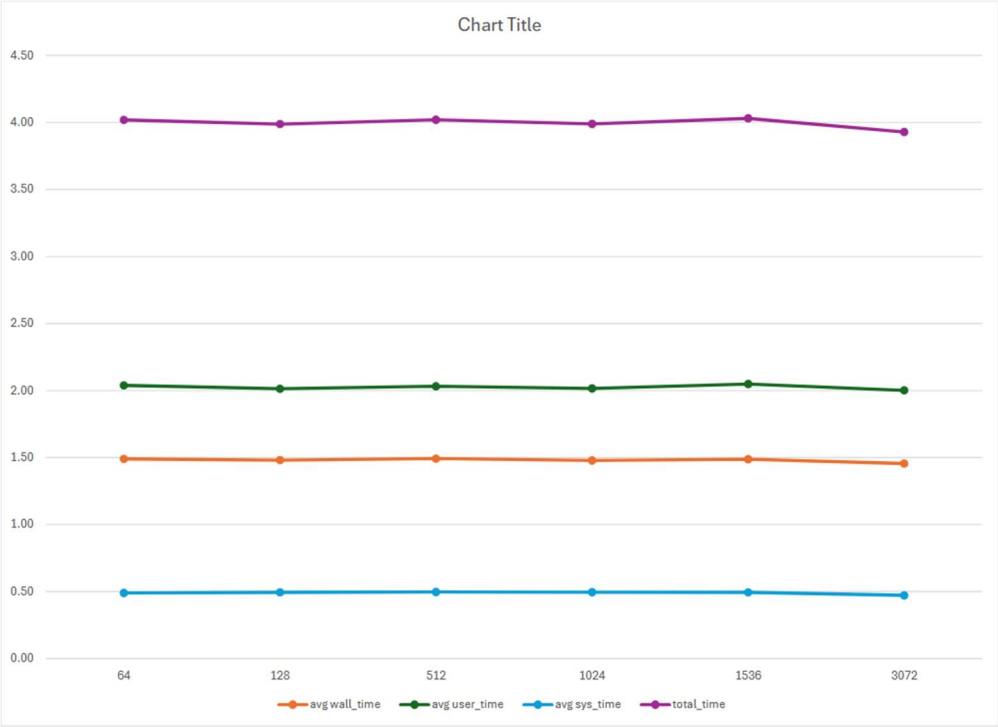
For this first part, the input file sized is fixed at 1.7 GB. We vary thread count and Slurm memory allocation. We repeat each configuration 3 times to obtain standard deviations to assess measurement noise.

3) Software Architecture: Pthreads

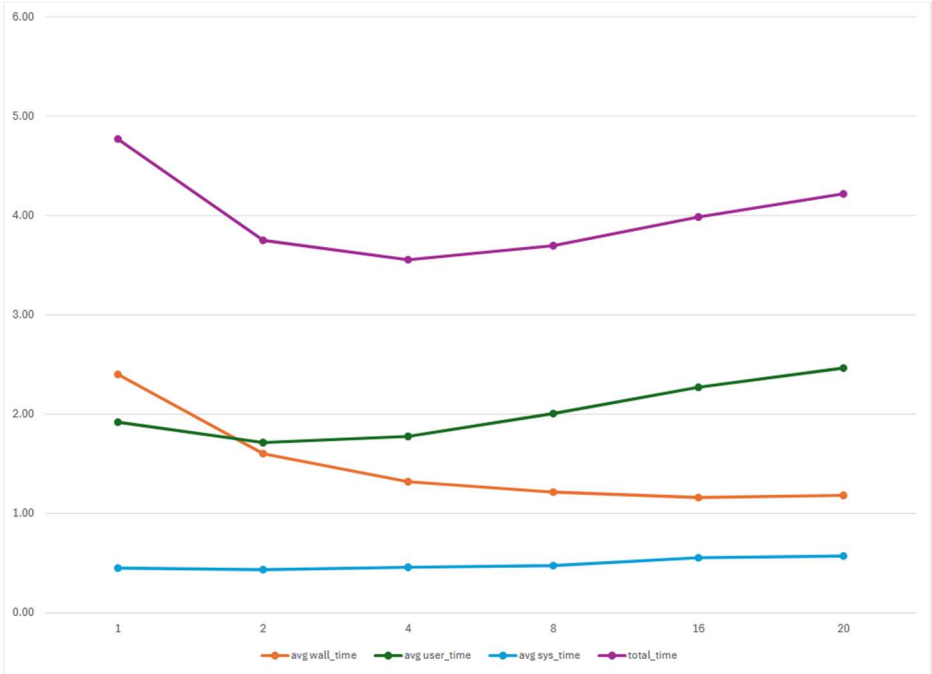
This design starts with N threads being crated once at start up and reused for every batch, avoiding repeated pthread_create overhead. The main thread reads lines in batches of up to 4096 using fgets(), signal workers, then waits for all processes to complete before printing results and loading the next batch.

4) Results: Pthreads

Memory Usage



Thread Count



5) Discussion

Pthreads

One of the biggest points of the pthreads implementation is that the workload is I/O bound at the application level. The main sequences `fgets()` loop serializes all file reads. For this small project we did not approach the memory budget. Threads beyond 4 threads applied diminishing returns, most of the benefit came from the first few threads.